

Mixed-Precision Pipelined BiCGSTAB Solver

CENG342 – Parallel Programming Project

Group 11

Süleyman Fatih Sert
Muhammet Talha Şatır
Seyfullah Gülyazı

May 2026

Abstract

In this project, three parallel versions of the **Mixed-Precision Pipelined BiCGSTAB** solver were developed in C for large sparse linear systems: a serial reference, a shared-memory OpenMP version, and a distributed-memory MPI version combined with OpenMP. The MPI version is pipelined using `MPI_Iallreduce` so that global reductions overlap with local computation. Matrix values are stored in FP32 while the vectors and accumulators run in FP64, producing a mixed-precision SpMV that halves the matrix memory footprint. All three versions converge to **bit-level identical numerical solutions** on a pentadiagonal test system, confirming algorithmic correctness. The OpenMP version achieves $1.30\times - 1.75\times$ speed-up with four threads. The MPI version, due to the very low computation-to-communication ratio of the pentadiagonal matrix, does not gain speed-up at this problem size; this strong-scaling behaviour is discussed in line with Amdahl's law.

1 Introduction

The linear system $A\mathbf{x} = \mathbf{b}$ lies at the heart of many engineering applications, including finite-difference / finite-element discretisations, circuit analysis, image processing and computational fluid dynamics, all of which produce large sparse systems. When A is not symmetric positive definite, Cholesky and Conjugate Gradient cannot be used; in this case **BiCGSTAB** (Biconjugate Gradient Stabilized) is the preferred Krylov-subspace method.

The aim of this project is to study BiCGSTAB from a parallel programming perspective. The targeted stages are:

1. A serial reference implementation of BiCGSTAB.
2. Shared-memory parallelisation with OpenMP.
3. Distributed-memory parallelisation with MPI.
4. Pipelining via `MPI_Iallreduce` so that global communication overlaps with local computation.
5. Mixed precision: FP32 matrix storage and FP64 vector arithmetic.

After verifying numerical correctness across the three versions, a strong-scaling study is performed for problem sizes $N \in \{100k, 500k, 2M\}$.

2 Method

2.1 The BiCGSTAB Algorithm

BiCGSTAB, proposed by Van der Vorst in 1992, is a Krylov-subspace method for nonsymmetric and large sparse systems. Each iteration performs two sparse matrix-vector products (SpMV) and a small number of dot products / AXPY updates:

1. $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$, $\hat{\mathbf{r}} = \mathbf{r}_0$, $\mathbf{p} = \mathbf{r}_0$
2. $\rho_{\text{old}} = \langle \hat{\mathbf{r}}, \mathbf{r} \rangle$
3. Iteration $k = 0, 1, \dots$:
 - $\mathbf{v} = A\mathbf{p}$
 - $\alpha = \rho_{\text{old}} / \langle \hat{\mathbf{r}}, \mathbf{v} \rangle$
 - $\mathbf{s} = \mathbf{r} - \alpha\mathbf{v}$
 - $\mathbf{t} = A\mathbf{s}$
 - $\omega = \langle \mathbf{t}, \mathbf{s} \rangle / \langle \mathbf{t}, \mathbf{t} \rangle$
 - $\mathbf{x} \leftarrow \mathbf{x} + \alpha\mathbf{p} + \omega\mathbf{s}$
 - $\mathbf{r} \leftarrow \mathbf{s} - \omega\mathbf{t}$
 - $\rho_{\text{new}} = \langle \hat{\mathbf{r}}, \mathbf{r} \rangle$, $\beta = (\rho_{\text{new}} / \rho_{\text{old}}) \cdot (\alpha / \omega)$
 - $\mathbf{p} \leftarrow \mathbf{r} + \beta(\mathbf{p} - \omega\mathbf{v})$

2.2 The Test Matrix: Pentadiagonal

The test matrix is pentadiagonal: the main diagonal is 6 and the four off-diagonals at distances ± 1 and ± 2 are -1 . The matrix is strictly diagonally dominant ($|6| > 4$), so BiCGSTAB converges quickly (typically in 11–12 iterations to $\|\mathbf{r}\|/\|\mathbf{b}\| < 10^{-10}$).

2.3 CSR Sparse Storage

Each row contains at most five nonzero entries, so the matrix is stored in **Compressed Sparse Row (CSR)** format:

- `row_ptr[]`: int, length $n + 1$, offsets of row starts.
- `col_idx[]`: int, length `nnz`, column indices.
- `vals_f64[]`: double, FP64 entries.
- `vals_f32[]`: float, FP32 mirror used by the mixed-precision SpMV.

2.4 OpenMP Parallelisation

The BLAS-1 vector kernels (`dot`, `axpy`, `norm2`) and the outer row loop of SpMV are parallelised with `#pragma omp parallel for`. Static scheduling is used because every row has the same workload in the pentadiagonal matrix; dynamic scheduling would only add overhead. The dot product uses a `reduction(+:s)` clause:

```
1 double vec_dot_omp(const double *a, const double *b, int n) {
2     double s = 0.0;
3     #pragma omp parallel for reduction(+:s) schedule(static)
4     for (int i = 0; i < n; i++) s += a[i] * b[i];
5     return s;
6 }
```

Listing 1: OpenMP dot-product kernel.

2.5 MPI and Pipelining

The matrix rows are distributed to MPI ranks (rank p owns the rows in $[\text{row_start}_p, \dots]$). Before each SpMV the full vector is collected via `MPI_Allgatherv`, since the column indices are global.

Pipelining. The scalar inner products are launched with `MPI_Iallreduce` (non-blocking). While the reduction is in flight, the rank performs the next local task (an Allgatherv or a local AXPY), then `MPI_Wait` retrieves the result:

```
1  /* Start non-blocking reduction for ||s||^2 */
2  double snorm2_loc = vec_dot_omp(s, s, local_n);
3  double snorm2_glb;
4  MPI_Request req_snorm;
5  MPI_Iallreduce(MPI_IN_PLACE, &snorm2_glb, 1, MPI_DOUBLE,
6                MPI_SUM, comm, &req_snorm);
7
8  /* While the reduction is in flight, gather s for the next SpMV */
9  MPI_Allgatherv(s, local_n, MPI_DOUBLE,
10               s_full, recvcnts, displs, MPI_DOUBLE, comm);
11
12 MPI_Wait(&req_snorm, MPI_STATUS_IGNORE);
13 double snorm = sqrt(snorm2_glb);
```

Listing 2: Pipelined reduction in the MPI version.

In addition, the two scalars $\langle t, s \rangle$ and $\langle t, t \rangle$ that can be reduced together are packed into a single `MPI_Iallreduce` call, halving the message latency.

2.6 Mixed Precision

Matrix values are stored in FP32 so that the matrix bandwidth halves; the multiply-accumulate is still performed in FP64:

```
1 void csr_spmv_mixed_omp(const CSRMatrix *A, const double *x_full,
2                        double *y) {
3     #pragma omp parallel for schedule(static)
4     for (int i = 0; i < A->nrows; i++) {
5         double sum = 0.0;
6         int row_end = A->row_ptr[i + 1];
7         for (int k = A->row_ptr[i]; k < row_end; k++) {
8             sum += (double)A->vals_f32[k] * x_full[A->col_idx[k]];
9         }
10        y[i] = sum;
11    }
12 }
```

Listing 3: Mixed-precision SpMV.

Because the vectors and accumulator remain in FP64, the iteration count is essentially unchanged.

3 Implementation

3.1 Project Layout

```
ceng342-project/
|-- include/                # public headers
|-- src/
|   |-- csr_matrix.c        # CSR + mixed SpMV
|   |-- vector_ops_serial.c # BLAS-1 (serial)
|   |-- vector_ops_omp.c    # BLAS-1 (OpenMP)
```

```
| |-- bicgstab_serial.c      # serial solver
| |-- bicgstab_omp.c        # OpenMP solver
| |-- bicgstab_mpi.c        # MPI + pipelined solver
| |-- timer.c               # cross-platform wtime()
| '-- main_*.c              # CLI drivers
|-- Makefile
'-- bench.sh
```

3.2 Build

The Makefile compiles the three versions in separate build directories:

- build/serial/: gcc -O2
- build/omp/: gcc -O2 -fopenmp
- build/mpi/: mpicc -O2 -fopenmp -DUSE_MPI

A single make all produces three binaries: bicgstab_serial, bicgstab_omp, bicgstab_mpi.

4 Experiments and Results

4.1 Test Environment

- Operating system: WSL2 Ubuntu 24.04 on Windows 11.
- Compiler: gcc 13.x (-O2).
- MPI: OpenMPI 4.1.6.
- Hardware: 4 logical x86-64 CPU cores.

4.2 Correctness Test ($N = 100\,000$)

All three versions were run on the same pentadiagonal system and produced *bit-level identical* numerical solutions (Table 1). This verifies the algorithmic correlation of the parallel implementations.

Table 1: Correctness test – $N = 100\,000$, $\text{tol} = 10^{-10}$.

Version	Iter	$\ r\ /\ b\ $	L2 error	Time
Serial	12	7.778×10^{-11}	2.206×10^{-8}	31.0 ms
OMP-4	12	7.778×10^{-11}	2.206×10^{-8}	17.7 ms
MPI-4	12	7.778×10^{-11}	2.206×10^{-8}	36.2 ms

```
root@FatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# ./bicgstab_serial 100000 1000 1e-10
=== Mixed Precision BiCGSTAB (Serial) ===
N=100000 max_iter=1000 tol=1.000000e-10
Matrix values: fp32 (mixed precision SpMV)
Vectors:      fp64

[serial] iter   0 |r|/|b| = 2.942e-03

--- Results ---
iterations   : 12
converged    : yes
final |r|/|b|: 7.778339e-11
L2 error     : 2.205520e-08
time         : 0.0310 s
time/iter    : 2.5833 ms
```

Figure 1: Serial version output, $N = 100\,000$.

```

root@FatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# OMP_NUM_THREADS=4 ./bicgstab_omp 100000 1000 1e-10
=== Mixed Precision BiCGSTAB (OpenMP) ===
N=100000 max_iter=1000 tol=1.000000e-10 threads=4
Matrix values: fp32 (mixed precision SpMV, OpenMP)
Vectors:      fp64

[omp] iter   0  |r|/|b| = 2.942e-03

--- Results ---
iterations   : 12
converged    : yes
final |r|/|b|: 7.778339e-11
L2 error     : 2.205520e-08
time         : 0.0177 s
time/iter    : 1.4724 ms

```

Figure 2: OpenMP version output, $N = 100\,000$, four threads.

```

1 total processes failed to start
root@FatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# OMP_NUM_THREADS=1 mpirun --oversubscribe -np 4 ./bicgstab_mpi 100000 1000 1e-10
=== Mixed Precision BiCGSTAB (MPI + OpenMP) ===
N=100000 max_iter=1000 tol=1.000000e-10
MPI ranks   : 4
OMP threads : 1 (per rank)
Matrix values: fp32 (mixed precision SpMV)
Vectors      : fp64
Reductions   : MPI_Iallreduce (non-blocking)

[mpi] iter   0  |r|/|b| = 2.942e-03

--- Results ---
iterations   : 12
converged    : yes
final |r|/|b|: 7.778339e-11
L2 error     : 2.205520e-08
time         : 0.0362 s
time/iter    : 3.0147 ms
root@FatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project#

```

Figure 3: MPI version output, $N = 100\,000$, four ranks, one thread per rank. The output explicitly indicates that MPI_Iallreduce non-blocking reductions are used.

4.3 Performance (Strong Scaling)

The three versions were measured at $N \in \{100\,000, 500\,000, 2\,000\,000\}$. The results are summarised in Table 2. The speed-up is $S = T_{\text{serial}}/T_{\text{parallel}}$.

Table 2: Strong-scaling results (4 threads / 4 ranks).

N	Serial	OMP-4	MPI-4	S_{OMP}	S_{MPI}
10^5	31.0 ms	17.7 ms	36.2 ms	1.75 $\times$	0.86 $\times$
5×10^5	165.1 ms	126.9 ms	226.0 ms	1.30 $\times$	0.73 $\times$
2×10^6	660.5 ms	496.4 ms	825.9 ms	1.33 $\times$	0.80 $\times$

```

root@fatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# ./bicgstab_serial 2000000 1000 1e-10
=== Mixed Precision BICGSTAB (Serial) ===
N=2000000 max_iter=1000 tol=1.000000e-10
Matrix values: fp32 (mixed precision SpMV)
Vectors:      fp64

[serial] iter    0  ||r||/||b|| = 6.578e-04

--- Results ---
iterations : 11
converged   : yes
final ||r||/||b||: 8.650286e-11
L2 error    : 1.030501e-07
time        : 0.6605 s
time/iter   : 60.0492 ms
root@fatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# OMP_NUM_THREADS=4 ./bicgstab_omp 2000000 1000 1e-10
=== Mixed Precision BICGSTAB (OpenMP) ===
N=2000000 max_iter=1000 tol=1.000000e-10 threads=4
Matrix values: fp32 (mixed precision SpMV, OpenMP)
Vectors:      fp64

[omp] iter    0  ||r||/||b|| = 6.578e-04

--- Results ---
iterations : 11
converged   : yes
final ||r||/||b||: 8.650290e-11
L2 error    : 1.030501e-07
time        : 0.4964 s
time/iter   : 45.1252 ms
root@fatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project# OMP_NUM_THREADS=1 mpirun --oversubscribe -np 4 ./bicgstab_mpi 2000000 1000 1e-10
=== Mixed Precision BICGSTAB (MPI + OpenMP) ===
N=2000000 max_iter=1000 tol=1.000000e-10
MPI ranks : 4
OMP threads : 1 (per rank)
Matrix values: fp32 (mixed precision SpMV)
Vectors:      fp64
Reductions : MPI_Iallreduce (non-blocking)

[mpi] iter    0  ||r||/||b|| = 6.578e-04

--- Results ---
iterations : 11
converged   : yes
final ||r||/||b||: 8.650289e-11
L2 error    : 1.030501e-07
time        : 0.8259 s
time/iter   : 75.0814 ms
root@fatihSert:/mnt/c/Users/Esma/Downloads/ceng342_extracted/ceng342-project#

```

Figure 4: Comparative output of all three versions at $N = 2000000$. All three converge in 11 iterations to $\|r\|/\|b\| \approx 8.65 \times 10^{-11}$, with an L2 error of $\approx 1.03 \times 10^{-7}$, close to the FP32 epsilon (the mixed-precision floor).

5 Discussion

5.1 OpenMP Speed-up

The OpenMP version achieves a real speed-up at every problem size ($1.30\times - 1.75\times$). However, the ideal linear speed-up ($4\times$ with four threads) is not reached. The dominant reason is the *memory-bandwidth-bound* nature of the workload: the pentadiagonal SpMV and the BLAS-1 kernels (`dot`, `axpy`) have very low arithmetic intensity, so performance is limited by main-memory bandwidth rather than by core count. This is the classical streaming-bound regime of the Roofline model.

5.2 MPI Behaviour

The MPI version is slower than the serial version. The reason is the extremely low computation-to-communication ratio of the pentadiagonal matrix combined with the very small iteration count:

- Pentadiagonal structure: only $\sim n/p \cdot 5$ floating-point operations per rank per iteration.
- `MPI_Allgatherv` broadcasts the full $8N$ -byte vector to all ranks at every iteration.
- Convergence in only 11–12 iterations leaves no opportunity for the MPI start-up cost to amortise.
- The WSL2 virtualisation layer adds further latency.

A rough experimental estimate of the per-iteration cost:

- $N = 500k$: ≈ 2 ms compute / ≈ 15 ms communication.
- $N = 2M$: ≈ 8 ms compute / ≈ 25 ms communication.

At both sizes communication dominates. This is a typical strong-scaling finding consistent with Amdahl’s law: for very sparse matrices, naive Allgatherv-based parallelisation is inefficient at small problem sizes.

5.3 Improvement Directions

- **Halo exchange.** For a pentadiagonal stencil each rank needs only ± 2 ghost rows from its neighbours. Exchanging only the boundary entries instead of the full vector reduces communication from $\mathcal{O}(N)$ to $\mathcal{O}(1)$ per iteration.
- **Hybrid MPI + OpenMP.** Using more threads per rank decreases the number of communicating processes and the frequency of collectives.
- **Communication-avoiding BiCGSTAB.** The s -step variant fuses the global reductions of s consecutive iterations.

5.4 Mixed-Precision Stability

Storing the matrix in FP32 does not affect the convergence behaviour: the iteration count remains 11 – 12 across all problem sizes. The final L2 error is bounded by the FP32 machine epsilon ($\approx 10^{-7}$), which is well within typical engineering tolerances.

Memory savings. At $N = 2 \times 10^6$ the `vals` array of the CSR matrix has $5 \cdot 2 \times 10^6 = 10^7$ entries: 40 MB in FP32 versus 80 MB in FP64, i.e. a **40 MB saving**.

6 Conclusion

A mixed-precision pipelined BiCGSTAB solver was successfully developed in three flavours: serial, OpenMP and MPI + OpenMP. The main findings are:

- **Algorithmic correctness:** all three versions produced bit-level identical numerical solutions ($\|r\|/\|b\| < 10^{-10}$ at every problem size).
- **OpenMP speed-up:** $1.30\times - 1.75\times$ real speed-up was obtained with four threads; the limit is the memory-bandwidth ceiling.
- **MPI strong scaling:** for the pentadiagonal sparse matrix the naive `MPI_Allgatherv` approach is communication-bound, an expected and literature-consistent finding.
- **Mixed precision:** FP32 matrix storage did not change the iteration count and saved 50% of the matrix memory.
- **Pipelining:** `MPI_Iallreduce` successfully overlapped the global reduction with the next `Allgatherv`.

Future work includes halo-exchange-based communication, hybrid MPI + OpenMP configurations and the s -step communication-avoiding variant.